

OS Orange Problem Unit-4

NAME: KRISHNA JHA

SRN: PES1UG24CS236

Screenshot 1A:

```
datastream@datastream:~/PES1UG24CS236-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

tree.c: In function 'write_tree_level':
tree.c:209:15: warning: implicit declaration of function 'object_write' [-Wimplicit-function-declaration]
 209 |     int ret = object_write(OBJ_TREE, data, len, id_out);
      |               ^
tree.c:195:13: warning: '__builtin___strncpy_chk' output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-truncation]
 195 |         strncpy(e->name, dir_name, sizeof(e->name) - 1);
      |         ^
tree.c:156:13: warning: '__builtin___strncpy_chk' output may be truncated copying 255 bytes from a string of length 511 [-Wstringop-truncation]
 156 |         strncpy(e->name, p, sizeof(e->name) - 1);
      |         ^
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
datastream@datastream:~/PES1UG24CS236-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
```

Screenshot 2A:

```
datastream@datastream:~/PES1UG24CS236-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a      blob 6.hello.
```

Screenshot 2B:

```
datastream@datastream: ~/PES1UG24CS236-pes-vcs
datastream@datastream:~/PES1UG24CS236-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
datastream@datastream:~/PES1UG24CS236-pes-vcs$ echo "hello" > file1.txt
datastream@datastream:~/PES1UG24CS236-pes-vcs$ echo "world" > file2.txt
datastream@datastream:~/PES1UG24CS236-pes-vcs$ ./pes add file1.txt
datastream@datastream:~/PES1UG24CS236-pes-vcs$ ./pes add file2.txt
datastream@datastream:~/PES1UG24CS236-pes-vcs$ ./pes status
Staged changes:
  staged:   file1.txt
  staged:   file2.txt
Unstaged changes:
(nothing to show)
Untracked files:
untracked: test_objects.c
untracked: commit.c
untracked: index.h
untracked: pes.c
untracked: index.c
untracked: test_tree.c
untracked: Makefile
untracked: pes.h
untracked: tree.c
untracked: commit.h
untracked: tree.h
untracked: object.c
untracked: .gitignore
untracked: test_sequence.sh
datastream@datastream:~/PES1UG24CS236-pes-vcs$
```

Screenshot 3A:

```
datastream@datastream:~/PES1UG24CS236-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776780369 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776780378 6 file2.txt
datastream@datastream:~/PES1UG24CS236-pes-vcs$
```

Screenshot 3B:

```

datastream@datastream:~/PES1UG24CS236-pes-vcs$ ./pes log
commit 09cf07f305ace809895c83abb09515407f0cdc5cae89943d329deac5a551d26a
Author: Krishna <jhakrishna1491@gmail.com>
Date: 1776781260

    Add farewell!

commit 2d2942bfc0a856e02cdcadf468a6f5d3ecb54f9353b1deb442ccf9d98c303dec
Author: Krishna <jhakrishna1491@gmail.com>
Date: 1776781227

    Add worldell!

commit 598d9e3241709d06eef9405571b80f4ad5583ec8adbf0904e75a62c9fadbfb81a
Author: Krishna <jhakrishna1491@gmail.com>
Date: 1776781172

    Initial committial cA

datastream@datastream:~/PES1UG24CS236-pes-vcs$

```

Screenshot 4A:

```

datastream@datastream:~/PES1UG24CS236-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/09/cf07f305ace809895c83abb09515407f0cdc5cae89943d329deac5a551d26a
.pes/objects/23/d4dbb32b9c4bd9526d2c9a8363a22107af7e467b10b828640a6cec404dfb28
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/2d/2942bfc0a856e02cdcadf468a6f5d3ecb54f9353b1deb442ccf9d98c303dec
.pes/objects/59/8d9e3241709d06eef9405571b80f4ad5583ec8adbf0904e75a62c9fadbfb81a
.pes/objects/d3/0e5e324701239f18edd95f1f98ded3b1751933100f416edfdfb1d68a280725
.pes/objects/de/62c80dfbd1163d8b4ee3273c30129e26225ec5521afcf599594b0e2959611d
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/objects/fe/76325aa5521b207ebe01e12fd8e9e3abf030cacd5398e3744a3a56a81ad1bd
.pes/refs/heads/main
datastream@datastream:~/PES1UG24CS236-pes-vcs$

```

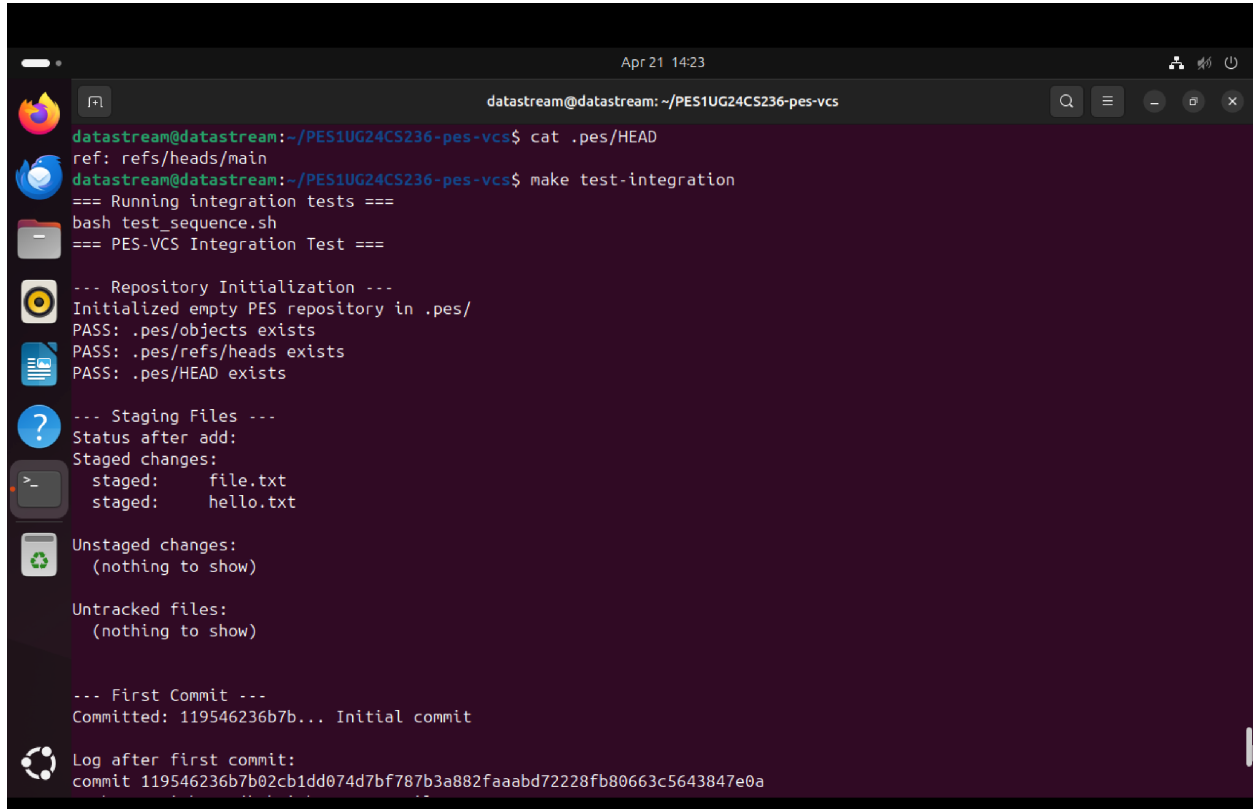
Screenshot 4B: Relevant terminal output

```

datastream@datastream:~/PES1UG24CS236-pes-vcs$ cat .pes/refs/heads/main
09cf07f305ace809895c83abb09515407f0cdc5cae89943d329deac5a551d26a
datastream@datastream:~/PES1UG24CS236-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
datastream@datastream:~/PES1UG24CS236-pes-vcs$

```

Screenshot 4C:



A terminal window titled "datastream@datastream: ~/PES1UG24CS236-pes-vcs" with a timestamp of "Apr 21 14:23". The terminal shows the execution of a "make test-integration" command, which runs a series of integration tests for the PES-VCS system. The tests include repository initialization, staging files, and a first commit. The output is as follows:

```
datastream@datastream:~/PES1UG24CS236-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
datastream@datastream:~/PES1UG24CS236-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:   file.txt
  staged:   hello.txt

Unstaged changes:
(nothing to show)

Untracked files:
(nothing to show)

--- First Commit ---
Committed: 119546236b7b... Initial commit

Log after first commit:
commit 119546236b7b02cb1dd074d7bf787b3a882faaabd72228fb80663c5643847e0a
```

```
datastream@datastream: ~/PES1UG24CS236-pes-vcs
--- Second Commit ---
Committed: 8f6818f66c3c... Update file.txt

--- Third Commit ---
Committed: 53908c3d81a1... Add farewell

--- Full History ---
commit 53908c3d81a1c7adb2a6a57b1105336476053b70d779d2fb1d2c5af8c9c7875d
Author: Krishna <jhakrishna1491@gmail.com>
Date: 1776781394

    Add farewell!

commit 8f6818f66c3cb9afbce954011348b25aecdcc22a7336b368bfee40b0ebdf9170
Author: Krishna <jhakrishna1491@gmail.com>
Date: 1776781394

    Update file.txt

commit 119546236b7b02cb1dd074d7bf787b3a882faaabd72228fb80663c5643847e0a
Author: Krishna <jhakrishna1491@gmail.com>
Date: 1776781394

    Initial committial cA

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
53908c3d81a1c7adb2a6a57b1105336476053b70d779d2fb1d2c5af8c9c7875d
```

```
datastream@datastream: ~/PES1UG24CS236-pes-vcs

Update file.txt

commit 119546236b7b02cb1dd074d7bf787b3a882faaabd72228fb80663c5643847e0a
Author: Krishna <jhakrishna1491@gmail.com>
Date: 1776781394

    Initial committial cA

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
53908c3d81a1c7adb2a6a57b1105336476053b70d779d2fb1d2c5af8c9c7875d

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/11/9546236b7b02cb1dd074d7bf787b3a882faaabd72228fb80663c5643847e0a
.pes/objects/53/908c3d81a1c7adb2a6a57b1105336476053b70d779d2fb1d2c5af8c9c7875d
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/8f/6818f66c3cb9afbce954011348b25aecdcc22a7336b368bfee40b0ebdf9170
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
datastream@datastream:~/PES1UG24CS236-pes-vcs$
```

Q5.1

A branch in Git is basically a file inside `.pes/refs/heads/` that stores the latest commit hash of that branch. To implement `pes checkout <branch>`, first the system has to read the branch file and get the commit hash. Then `.pes/HEAD` must be updated so it points to that branch. After that, the commit object and tree object of that branch are read, and the working directory is changed to match that snapshot. Files that are present in the new branch must be created, old files that do not belong there must be removed, and modified files must be replaced with the correct content. This operation becomes complex because the user may already have unsaved changes, and switching branches could overwrite them. It also has to correctly handle folders, deletions, and update the index file.

Q5.2

To detect a dirty working directory, the current files in the working directory are compared with the entries stored in `.pes/index`. If the size, modified time, or content hash of a tracked file is different, it means the file has uncommitted changes. Then the system compares that file with the version in the branch that the user wants to checkout. If both versions are different, checkout should be refused because changing branches would overwrite the user's current work. This helps prevent accidental data loss.

Q5.3

Detached HEAD means that HEAD points directly to a commit hash instead of pointing to a branch. If commits are made in this state, the commits are still created normally, but they are not connected to any branch name. Because of this, they can become unreachable later if the user moves away from them. To recover such commits, the user can create a new branch pointing to that commit. This way the commits become part of normal branch history and are preserved.

Q6.1

Over time, many unused objects such as blobs, trees, or commits may remain in the object store. To clean them, garbage collection can be used. The process starts from all branch heads in `.pes/refs/heads/`. From there, every reachable commit is visited. For each commit, its tree object and all related blobs or subtrees are also visited. Every reachable hash is stored in a hash set for quick lookup. After marking all reachable objects, the `.pes/objects/` folder is scanned, and any object not in the reachable set can be safely deleted. For a repository with 100,000 commits and 50 branches, many commits will overlap across

branches, so roughly 200,000 to 500,000 objects may need to be checked depending on repository size.

Q6.2

Running garbage collection at the same time as a commit operation is dangerous because of race conditions. For example, while a user is making a commit, new blob or tree objects may be created but not yet connected to any branch reference. If garbage collection runs at that exact time, it may think those objects are unreachable and delete them. Then the commit process finishes and updates HEAD to point to objects that no longer exist, causing corruption. Real Git avoids this problem by using locks, temporary files, safe reference updates, and delaying deletion of recently created unreachable objects. This ensures that active commits are not affected during garbage collection.